

SIMATS ENGINEERING

ASSESSMENT TOOL 3: REVERSE ENGINEERING TASK

Cloud Computing and Big Data Analytics – CSA1558

CO4: Analyze big data characteristics and challenges across domains including security and application aspects.

Question Count: 5 | Total Marks: 30

Student Name : T . Kodanda Ram
Register Number : 192524162

Question 1: Website and Customer Transaction Big Data Platform

The platform collects billions of website clicks, mobile interactions, and customer transactions every day. Because the data volume is extremely large and comes from structured and semi-structured sources, a distributed Big Data architecture is likely required.

1. Understanding of the Given System

A probable architecture is:

Probable Architecture

Web / Mobile / Transactions	Data Ingestion	Distributed Storage	Batch+Stream Processing	Analytics	BI Dashboards
↓	↓	↓	↓	↓	

2. Probable Distributed Storage System

A distributed storage system such as HDFS or cloud object storage is likely to be used. Structured transaction data can be stored in structured tables or a data warehouse, while semi-structured clickstream and mobile-event data can be stored in a data lake. Replication would probably protect data from node failures, and partitioning could be based on date, region, or event type.

3. Probable Data Ingestion Framework

A distributed event-ingestion framework such as Apache Kafka is a likely choice for continuously receiving website clicks, mobile interactions, and customer transactions. Kafka partitions can support parallel ingestion.

4. Probable Batch and Stream Processing

Apache Spark is likely for large-scale batch analytics, while Spark Structured Streaming or a similar streaming engine is likely for near-real-time analytics.

5. Structured and Semi-Structured Data

Data	Probable Storage
Customer transactions	Structured database / data warehouse
Website clicks	Data lake / semi-structured storage
Mobile events	Data lake
Analytical datasets	Data warehouse / analytical store

6. Scalability and Partitioning

- Horizontal scaling by adding storage and processing nodes.
- Kafka partitions for parallel event ingestion.
- Data partitioning by date, region, or event type.
- Replication for fault tolerance.
- Distributed processing to divide large workloads among machines.

7. Dashboard and BI Integration

The analytics layer would likely feed a data warehouse or analytical database, which can then connect to Business Intelligence dashboards. Dashboards could display customer activity, transaction trends, and business KPIs.

8. Reverse-Engineering Conclusion

The massive data volume and continuous event generation strongly suggest a distributed architecture. Kafka for ingestion, HDFS/cloud object storage for distributed storage, Spark for batch and streaming analytics, and a warehouse/BI layer are probable components inferred from the requirements.

Question2:E-Commerce PersonalizationPlatform

The e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. It requires real-time personalization, customer profiling, recommendations, and KPI monitoring.

Probable E-Commerce Architecture

User Events	Event Streaming	Stream Processing	CustomerProfiles / Features	Recommendation ML	Personalized Output / KPIs
↓	↓	↓	↓	↓	

1. Probable Event Streaming Tools

ApacheKafka is a likely event-streaming platform. Separate topics could be used for product views, cart additions, checkout events, and payment events. Partitions would allow parallel processing across regions.

2. Customer Profiling Database

A NoSQL database is likely for rapidly changing customer profiles containing viewed products, cart history, purchase history, preferences, and recent activity. Structured order and payment records could be maintained separately.

3. Real-Time Synchronization

Session data, clickstream logs, and transaction records can be synchronized through the streaming layer:

Click/Product View → Kafka → Stream Processing → Customer Profile Update → Recommendation Model → Personalized Recommendation

4. Probable Caching and Query Systems

A fast in-memory cache such as Redis is likely for active sessions, frequently accessed products, recent preferences, and recommendations. A distributed query engine such as Presto/Trino could query large datasets across distributed storage.

5. Machine Learning Workflow

A probable recommendation workflow is: Historical Data → Feature Engineering → Model Training → Recommendation Model → Real-Time Customer Features → Recommendation. Collaborative filtering, content-based, or hybrid approaches are possible.

6. Structured and Semi-Structured Data

Data	Examples
Structured order data	Order ID, customer ID, product ID, price, payment status
Semi-structured browsing data	Clicks, product views, searches, session events

A distributed processing layer can join these datasets using identifiers such as customer ID and product ID.

7. Scalability and Fault Tolerance

- ☐ Kafka partitions for parallel event processing.
- ☐ Replication for fault tolerance.
- ☐ Distributed databases for large customer profiles.
- ☐ Multiple processing nodes.
- ☐ Caching to reduce database load.
- ☐ Regional processing to reduce latency.

8. Dashboard and KPI Monitoring

Processed data can feed dashboards showing conversion rate, cart abandonment, product views, purchase rate, recommendation engagement, and regional performance.

9. Reverse-Engineering Conclusion

The real-time personalization requirement strongly suggests event streaming, distributed processing, customer-profile storage, caching, and machine learning. Kafka, Spark, NoSQL, Redis, and a distributed query engine are probable components.

Question 3: Healthcare Big Data System

The healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. It connects hospitals, laboratories, and insurance systems and supports disease prediction, monitoring, clinical decision-making, and population health studies.

Probable Healthcare Big Data Architecture

Hospitals / Labs / Wearables	Integration / ETL	Hybrid Storage	Stream + Batch Processing	ML / Analytics	Clinical / Population Insights
↓	↓	↓	↓	↓	

1. Probable Hybrid Database Architecture

Data Type	Probable Storage
Patient records	Relational database
Prescription records	Relational database
Diagnostic images	Object / distributed storage
Wearable streams	Time-series / NoSQL storage
Analytics datasets	Data lake / data warehouse

The hybrid design is inferred from the different structures and access patterns of the healthcare data.

2. Probable Data Integration Tools

An integration/ETL layer is likely to extract data from hospitals, laboratories, and insurance systems, transform it into common formats, validate it, remove duplicates, and load it into the analytics platform. HL7/FHIR are probable candidates for healthcare information exchange, although the case does not mandate a specific standard.

3. Streaming and Machine Learning

Wearable devices generate continuous streams. A probable flow is Wearables → Stream Ingestion → Stream Processing → Monitoring. Machine-learning frameworks can use historical and current data for disease-risk prediction and monitoring.

4. Privacy, Encryption and Compliance

- ☐ Encryption during transmission and storage.
- ☐ Strong authentication and role-based access control.
- ☐ Audit logs for sensitive-data access.
- ☐ Data minimization and controlled sharing.
- ☐ Monitoring and governance mechanisms.

The case requires privacy, encryption, and compliance, but it does not name a specific regulation; therefore no particular regulatory framework is assumed here.

5. Analytics Pipeline

Patient Data → Data Integration → Quality Validation → Storage → Distributed Processing → ML/Analytics → Clinical Decision Support

For population studies: Historical Patient Data → Distributed Processing → Statistical/ML Analysis → Population Health Insights.

6. Reverse-Engineering Conclusion

The combination of structured records, unstructured images, streaming wearable data, and multi-organization integration strongly indicates hybrid storage, an integration layer, streaming/batch processing, ML analytics, and strong privacy controls.

Question 4: Smart City Big Data Platform

The smart-city platform continuously receives traffic sensor feeds, CCTV metadata, weather updates, and public transport activity. It requires real-time congestion prediction and batch analytics for urban planning.

Probable Smart City Architecture

Sensors / CCTV / Weather / Transport	Edge Computing	Distributed Messaging	Stream + Batch Analytics	Time-Series / Geospatial Storage	Live Operations Dashboard
↓	↓	↓	↓	↓	

1. Probable Edge Computing Setup

Edge computing is likely near traffic sensors and other city devices to collect data, perform basic filtering, detect immediate events, and reduce the amount of data transmitted to centralized systems.

2. Probable Distributed Messaging

ApacheKafkaisaprobablemessagingframework. Traffic, weather, public transport, and CCTV metadata canbeseparatedintotopics,withpartitionssupporting parallel consumption.

3. Centralized and Specialized Storage

Data / Requirement	Probable Technology
Large historical datasets	Distributed data lake / cloud object storage
Sensor measurements over time	Time-series database
Location-based queries	Geospatial database
Historical analytical reporting	Data warehouse

4. Geospatial and Streaming Analytics

Streaminganalyticscanidentifycongestion,sudden traffic changes, public transport delays, and weather-related effects. Geospatial processing can associate events with roads, intersections, routes, and geographic regions.

5. Batch and Real-Time Analytics

Real-time: Traffic Stream → Stream Processing → Congestion Detection → Live Operations Dashboard

Batch: Historical Traffic + Weather + Transport Data → Distributed Processing → Urban Planning Analytics

6. Security and Access Control

- ☐ Authentication and role-based access control.
- ☐ Encryption of sensitive data.
- ☐ Network security controls.
- ☐ Access logging and monitoring.
- ☐ Separation of public, operational, and restricted information.

7. Reverse-Engineering Conclusion

Continuous sensor feeds strongly suggest edge computing, distributed messaging, stream processing, and specialized time-series and geospatial storage. Combining real-time and batch analytics supports both immediate city operations and long-term planning.

Question 5: Healthcare Analytics System

The healthcare analytics system integrates patient records, wearable readings, laboratory reports, and appointment histories. It requires hybrid storage, interoperability, real-time monitoring, predictive analytics, distributed processing, and strong data governance.

Probable Healthcare Analytics Architecture

Hospitals / Clinics / Labs / Wearables	ETL + Interoperability	HybridStorage	Stream + Distributed Batch Processing	ML Analytics	Risk Scoring / Recommendations
↓	↓	↓	↓	↓	

1. Probable Hybrid Storage Architecture

Data	Probable Storage
Patient records	Relational database
Appointment history	Relational database
Laboratory reports	Structured / semi-structured storage

Wearable readings	Time-series / NoSQL storage
Large analytics datasets	Distributed data lake

2. Probable ETL Pipeline

Extract → Validate → Transform → Standardize → Integrate → Load

The pipeline can extract data from hospitals, clinics, laboratories, and devices; standardize formats; validate quality; remove duplicates; integrate records; and load the resulting datasets for analysis.

3. Interoperability

Standardized healthcare data exchange is likely required. HL7/FHIR are probable interoperability standards, although the assessment does not explicitly mandate a particular standard.

4. Real-Time Monitoring

Wearable → Stream Ingestion → Stream Processing → Patient Monitoring → Alert

Continuous wearable readings can be processed to identify abnormal measurements and support timely monitoring.

5. Predictive Health Analytics

Historical Data → Feature Engineering → ML Model → Risk Score → Clinical Support

Classification, regression, or anomaly-detection approaches may support risk prediction. The exact model cannot be determined from the case alone.

6. Distributed Processing

A distributed framework such as Apache Spark is likely for population-level analysis. Large patient datasets can be divided across multiple processing nodes to improve scalability and reduce processing time.

7. Encryption, Compliance and Auditing

- ☐ Encryption at rest and in transit.
- ☐ Authentication and role-based authorization.
- ☐ Audit logs and access monitoring.
- ☐ Data retention and governance policies.
- ☐ Controlled access to sensitive medical information.

The case requires compliance but does not specify a particular regulation; therefore no specific regulatory framework is claimed as mandatory.

8. Machine Learning Support

ML may support diagnosis-risk scoring, early warning, patient risk stratification, and treatment-support recommendations. These outputs should be treated as analytical support rather than autonomous medical decisions.

9. Reverse-Engineering Conclusion

The mixture of structured medical records, wearable streams, laboratory information, and appointment history strongly indicates hybrid storage with ETL/interoperability, real-time streaming, distributed processing, and ML analytics. Encryption, access control, and auditing are likely because the system handles sensitive medical information.

Rubric Alignment – Level 4 Target

The answers are organized to address the Reverse Engineering Task rubric in the uploaded assessment:

Criterion	How the answers address it
Inference Accuracy – 25%	Identifies probable components and data flows from clues in each case rather than treating inferred technologies as stated facts.
Understanding of Architecture – 25%	Explains storage, ingestion, processing, analytics, ML, integration, and output layers.
Use of Technical Evidence – 20%	Provides technical reasons for likely technologies, partitioning, streaming, replication, storage, and security choices.
Depth of Analysis – 15%	Explains design decisions, scalability, fault tolerance, data structures, and operational implications.
Clarity – 15%	Uses structured headings, tables, architecture flows, numbered steps, and concise explanations.

Important: The assessment describes systems but does not explicitly name all technologies. Therefore, technologies such as Kafka, Spark, HDFS/cloud object storage, NoSQL, Redis, and distributed query engines are presented as probable or inferred components, not as confirmed facts.